Week 11: Connecting an application

Learning outcomes: CLO4, CLO6

Lesson

A **three-tier** application has a presentation layer (what the user sees), an application layer (the code and its rules) and a data layer (the database). In this course you write the application layer in Python or Node.js.

Read connection details from environment variables, not from the code, and connect with an account that has only the permissions the app needs.

SQL injection happens when user input is pasted into a SQL string, for example `"... WHERE province_pcode = '" + code + "'"`. Input such as `' OR '1'='1` then changes the meaning of the query. A **parameterized query** sends the SQL and the values separately (`%s` in Python, `?` in Node.js), so the input is always treated as a value. Parameters can replace values only, not table or column names.

Your code should handle empty results and connection errors, and always close the connection.

Worked example

Run `example.sql` (see Run a SQL file (`setup/README.md`)); it shows a prepared statement inside MySQL. Then run the report app from apps/ (`apps/README.md`) and put your own code in this week's app/ (`weeks/week-11/app/README.md`) folder.

Tables for this week

Read from the `cambodia` database and make your changes in `student_work`. PK means primary key and FK means foreign key. Every column is `NOT NULL`. The sample rows are the first two rows of each table, sorted by code.

`cambodia.province`

Column	Type	Key	Meaning
`province_pcode`	`CHAR(4)`	PK	Province code
`province_name`	`VARCHAR(100)`		Province name (English)
`area_km2`	`DECIMAL(12,1)`		Area in km², one decimal place

Sample rows:

province_pcode	province_name	area_km2
KH01	Banteay Meanchey	6145.9
KH02	Battambang	11869.0

`cambodia.district`

Column	Type	Key	Meaning
`district_pcode`	`CHAR(6)`	PK	District code
`district_name`	`VARCHAR(100)`		District name (English)
`area_km2`	`DECIMAL(12,1)`		Area in km², one decimal place
`province_pcode`	`CHAR(4)`	FK	Province code

Sample rows:

district_pcode	district_name	area_km2	province_pcode
KH0102	Mongkol Borei	807.2	KH01
KH0103	Phnum Srok	761.7	KH01

Join keys: `district.province_pcode = province.province_pcode`. Join on the codes, not the names.

The report app: give it a `province_pcode` such as `KH01`. It queries `cambodia.district` and returns `district_pcode` and `district_name`, sorted by district code. It only reads data; create your own table in `student_work` for the CRUD tasks.

Run `DESCRIBE cambodia.province;` to see a table definition in MySQL. The data guide (`data/README.md`) explains where the data comes from.

Lab tasks

1. Choose Python or Node.js and run the district report app for one province. Save the command and the output.
2. Find where the app binds its parameter. Explain how this prevents SQL injection and why column names cannot be parameters.
3. Run the report with the input `' OR '1'='1` and check that it returns no rows.
4. In your own app, create a table in `student_work`, then insert rows copied from `cambodia`, select them, update one and delete one, using parameters for every value.
5. Make your app handle a failed connection (for example, a wrong port) with a clear message, and always close the connection. Optional: rank the districts in each province by area with `ROW_NUMBER()`, using the code as a tie breaker.

Homework

1. Submit your app with short setup instructions and the output of a run.
2. Explain how parameters, permissions and keeping passwords out of code each protect the app.

Capstone sprint

C11 / Checkpoint 3: Deliver a working app with your schema, import script, database objects, queries and output. A classmate should be able to run it from a fresh clone of your repository.

Submit

Write your answers in `work.sql` and label each one, for example `-- Lab 3`. Answer the questions in `reflection.md` and save your query output in `results/`. Then commit and push to your repository. See SUBMISSIONS.md for details.